

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ
ПРОГРАММИРОВАНИЕ»
Вариант №9

Выполнила:

Ковжого Елизавета Андреевна

3 курс, группа ИВТ-б-о-24-1,

09.03.01 «Информатика и вычислительная техника», направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Р. А., доцент
департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии.

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Элементы объектно-ориентированного программирования в языке Python

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x

Порядок выполнения работы:

Адрес созданного репозитория:

1. Создали репозиторий и выполнили клонирование репозитория.
2. Проработали примеры представленные в лабораторной работе.

Листинг кода pr1.py:

```
class Pet:
```

```
    kind = 'mammal'
```

```
    n_pets = 0
```

```
    pet_names = []
```

```
    def __init__(self,spec,name):
```

```
        self.name = name
```

```
        self.spec= spec
```

```
        self.leg = 4
```

```
tom = Pet("cat","Tom")
```

```
avacado = Pet('dog','Avacado')
```

```
ben = Pet('goldfish','Ben')
```

```
Pet.n_pets+=3
```

```
print(Pet.n_pets)
```

```
print(tom.n_pets)
```

```
print(avacado.n_pets)
```

```
print(ben.n_pets)
```

```
ben.kind = 'fish'
```

```
print(Pet.kind)
```

```
print(tom.kind)
```

```
print(avacado.kind)
```

```
print(ben.kind)
```

```
tom.pet_names.append(tom.name)
```

```
avacado.pet_names.append(avacado.name)
```

```
ben.pet_names.append(ben.name)
```

```
print(tom.pet_names)
```

```
tom.pet_names.append("Tom")
```

```
avacado.pet_names.append("Avacado")
```

```
ben.pet_names.append("Ben")
```

```
print(tom.pet_names)
```

```
print(avacado.pet_names)
```

```
print(ben.pet_names)
```

```
ben.leg =0
```

```
Pet.all_specs = [tom.spec,avacado.spec,ben.spec]
```

```
avacado.bred = 'corgi'
```

Листинг кода pr2.py:

```
class MyClass:
```

```
    def __init__(self,arg):
```

```
        self.arg = arg
```

```
def do_smt(self):  
    print('do something')  
  
my_object = MyClass(10)  
my_object.do_smt()  
MyClass.do_smt(my_object)
```

Листинг кода pr3.py:

```
class Ship:  
    def __init__(self,name,capacity):  
        self.name = name  
        self.capacity = capacity  
        self.cargo =0  
    def sail(self):  
        print("{} has sailed!".format(self.name))  
    def convert_cargo(self):  
        cargo_kg = self.cargo*1000  
        return cargo_kg  
    def name_captain(self,cap):  
        self.captain = cap  
        print("{} is the captain of the {}".format(self.captain,self.name))  
    def load_cargo(self, weight):  
        if self.cargo+weight < self.capacity:  
            self.cargo += weight  
            print("Loaded {} tons!".format(weight))  
        else:  
            print("Cannot load that much")  
    def unload_cargo(self,weight):  
        if self.cargo-weight >= 0:
```

```

        self.cargo -= weight
        print("Unloaded {} tons!".format(weight))
    else:
        print("Cannot unload that much")
def free_space(self):
    return self.cargo - self.cargo

def sail_func(name):
    print("{} has sailed!".format(name))

black_pearl = Ship("Black Pearl",800)
black_pearl.sail()

sail_func(black_pearl.name)

print(black_pearl.convert_cargo())

black_pearl.name_captain("Jask Sparrow")
print(black_pearl.name_captain)

black_pearl.load_cargo(600)
black_pearl.unload_cargo(400)
black_pearl.load_cargo(700)
black_pearl.unload_cargo(300)

```

Листинг кода pr4.py:

```

class Rectangle:
    def __init__(self, width, height):
        self.__width = width
        self.__height = height
    @property
    def get_width(self):
        return self.__width
    def get_height(self):
        return self.__height
    def area(self):
        return self.__width * self.__height
    def set_width(self, width):
        self.__width = width
    def set_height(self, height):
        self.__height = height

rect = Rectangle(10,20)
print(rect.get_width(), rect.get_height())
print(rect._Rectangle__width)
rect._Rectangle__width=20

```

Листинг кода pr5.py:

```

class Rectangle:
    def __init__(self, width, height):
        self.__width = width
        self.__height = height
    @property
    def width(self):
        return self.__width

```

```

@width.setter
def width(self, w):
    if w > 0:
        self.__width = w
    else:
        raise ValueError("width must be positive")

@property
def height(self):
    return self.__height

@height.setter
def height(self, h):
    if h > 0:
        self.__height = h
    else: raise ValueError("height must be positive")

def area(self):
    return self.__width * self.__height

```

```

rect = Rectangle(10,20)
rect.width = 100
rect.height = 200
print(rect.width(), rect.height())

```

Листинг кода pr6.py:

```

#!/usr/bin/env python3
#-*- coding: utf-8 -*-

class Rational:
    def __init__(self, a=0,b=1):

```

```
a = int(a)
b = int(b)
if b==0:
    raise ValueError()
self.__numerator = abs(a)
self.__denominator = abs(b)
```

```
self.__reduce()
```

```
def __reduce(self):
    def gcd(a,b):
        if a ==0:
            return b
        elif b==0:
            return a
        elif a>=b:
            return gcd(a%b,b)
        else:
            return gcd(a, b%a)
```

```
c = gcd(self.__numerator, self.__denominator)
self.__numerator //=c
self.__denominator //=c
```

```
@property
def numerator(self):
    return self.__numerator

@property
def denominator(self):
```

```
return self.__denominator
```

```
def read(self, prompt=None):
```

```
    line = input() if prompt is None else input(prompt)
```

```
    parts = list(map(int, line.split('/', maxsplit=1)))
```

```
    if parts[1]==0:
```

```
        raise ValueError()
```

```
    self.__numerator = abs(parts[0])
```

```
    self.__denominator = abs(parts[1])
```

```
    self.__reduce()
```

```
def display(self):
```

```
    print(f"{self.__numerator}/{self.__denominator}")
```

```
def add(self, rhs):
```

```
    if isinstance(rhs, Rational):
```

```
        a = self.numerator * rhs.denominator + \
```

```
            self.denominator * rhs.numerator
```

```
        b = self.denominator * rhs.denominator
```

```
        return Rational(a,b)
```

```
    else:
```

```
        raise ValueError()
```

```
def sub(self, rhs):
```

```
    if isinstance(rhs, Rational):
```

```
        a = self.numerator * rhs.denominator - \
```

```
            self.denominator * rhs.numerator
```

```

        b = self.denominator * rhs.denominator

    return Rational(a, b)
else:
    raise ValueError()

def mul(self, rhs):
    if isinstance(rhs, Rational):
        a=self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator
        return Rational(a,b)
    else:
        raise ValueError()

def div(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator

        return Rational(a, b)
    else:
        raise ValueError()

def equals(self, rhs):
    if isinstance(rhs, Rational):
        return (self.numerator == rhs.numerator) and \
            (self.denominator == rhs.denominator)
    else:
        return False

```

```

def greater(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator

        return v1 > v2
    else:
        return False

def less(self, rhs):
    if isinstance(rhs, Rational):
        v1 = self.numerator / self.denominator
        v2 = rhs.numerator / rhs.denominator

        return v1 < v2
    else:
        return False

if __name__ == '__main__':
    r1 = Rational(3, 4)
    r1.display()

    r2 = Rational()
    r2.read("Введите обыкновенную дробь: ")
    r2.display()

    r3 = r2.add(r1)
    r3.display()

```

```
r4 = r2.sub(r1)
```

```
r4.display()
```

```
r5 = r2.mul(r1)
```

```
r5.display()
```

```
r6 = r2.div(r1)
```

```
r6.display()
```

3. Выполнили предложенные задания по варианту

Задание №1:

Парой называется класс с двумя полями, которые обычно имеют имена `first` и `second`. Требуется реализовать тип данных с помощью такого класса. Во всех заданиях обязательно должны присутствовать: метод инициализации `__init__` ; метод должен контролировать значения аргументов на корректность; ввод с клавиатуры `read`; вывод на экран `display`.

Реализовать внешнюю функцию с именем `make_тип()` , где `тип` — тип реализуемой структуры. Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу:

9. Поле `first` — целое положительное число, часы; поле `second` — целое положительное число, минуты. Реализовать метод `minutes()` — приведение времени в минуты.

Листинг кода `z1.py`:

```
#!/usr/bin/env python3  
# -*- coding: utf-8 -*-
```

```
class Pair:
```

```
def __init__(self, first=0, second=0):
    if first < 0 or second < 0:
        raise ValueError("Аргументы должны быть положительными
числами.")
    if second >= 60:
        raise ValueError("Количество минут не может быть больше или
равно 60.")
    self.first = first
    self.second = second

def read(self,):
    line = input()
    parts = list(map(int, line.split()))
    if len(parts) != 2:
        raise ValueError("Необходимо ввести два числа через пробел.")
    if parts[0] < 0 or parts[1] < 0:
        raise ValueError("Числа должны быть положительными.")
    if parts[1] >= 60:
        raise ValueError("Количество минут не может быть больше или
равно 60.")
    self.first = parts[0]
    self.second = parts[1]

def display(self):
    print(f"Часы: {self.first}, Минуты: {self.second}")

def minutes(self):
    return self.first * 60 + self.second
```

```
def make_time(first, second):  
  
    try:  
        return Pair(first, second)  
    except (ValueError, TypeError) as e:  
        print(e)  
        exit(1)
```

```
if __name__ == '__main__':
```

```
    print("\n make_time:")  
    time1 = make_time(2, 30)  
    time1.display()  
    print(f"Время в минутах: {time1.minutes()}")
```

```
    print("\nконструктор Pair:")  
    time2 = Pair(1, 15)  
    time2.display()  
    print(f"Время в минутах: {time2.minutes()}")
```

```
    print("\nввод данных с клавиатуры:")  
    time3 = Pair()  
    time3.read()  
    time3.display()  
    print(f"Время в минутах: {time3.minutes()}")
```

```
make_time:
Часы: 2, Минуты: 30
Время в минутах: 150

конструктор Pair:
Часы: 1, Минуты: 15
Время в минутах: 75

ввод данных с клавиатуры:
12 59
Часы: 12, Минуты: 59
Время в минутах: 779
```

Рис. 1 — Выполнение программы

Задание №2:

Составить программу с использованием классов и объектов для решения задачи. Во всех заданиях, помимо указанных в задании операций, обязательно должны быть реализованы следующие методы: метод инициализации `__init__` ; ввод с клавиатуры `read` ; вывод на экран `display` .

9. Реализовать класс `Account`, представляющий собой банковский счет. В классе должны быть четыре поля: фамилия владельца, номер счета, процент начисления и сумма в рублях. Открытие нового счета выполняется операцией инициализации. Необходимо выполнять следующие операции: сменить владельца счета, снять некоторую сумму денег со счета, положить деньги на счет, начислить проценты, перевести сумму в доллары, перевести сумму в евро, получить сумму прописью (преобразовать в числительное).

Листинг кода `z2.py`:

```
#!/usr/bin/env python3

# -*- coding: utf-8 -*-
```

```

class Account:
    def __init__(self, surname="", account_number="", interest_rate=0.0,
balance=0.0):
        if not isinstance(balance, (int, float)) or balance < 0:
            raise ValueError("Сумма на счете не может быть отрицательной.")
        if not isinstance(interest_rate, (int, float)) or interest_rate < 0:
            raise ValueError("Процент начисления не может быть
отрицательным.")

        self.surname = surname
        self.account_number = account_number
        self.interest_rate = interest_rate
        self.balance = float(balance)

    def read(self):
        print( "Введите данные счета:")
        self.surname = input("Фамилия владельца: ")
        self.account_number = input("Номер счета: ")
        try:
            self.interest_rate = float(input("Процент начисления: "))
            self.balance = float(input("Сумма в рублях: "))
            if self.balance < 0 or self.interest_rate < 0:
                raise ValueError
        except ValueError:
            print("Ошибка: введены некорректные числовые данные.")
            exit(1)

    def display(self):

```

```
print(f"Владелец: {self.surname}")
print(f"Номер счета: {self.account_number}")
print(f"Процент начисления: {self.interest_rate}%")
print(f"Баланс: {self.balance:.2f} руб.")
```

```
def change_owner(self, new_surname):
    if not new_surname:
        raise ValueError("Фамилия не может быть пустой.")
    self.surname = new_surname
    print(f"Владелец счета изменен на: {self.surname}")
```

```
def withdraw(self, amount):
    if amount <= 0:
        raise ValueError("Сумма снятия должна быть положительной.")
    if amount > self.balance:
        print("Недостаточно средств на счете.")
        return False
    self.balance -= amount
    print(f"Снято {amount:.2f} руб. Остаток: {self.balance:.2f} руб.")
    return True
```

```
def deposit(self, amount):
    if amount <= 0:
        raise ValueError("Сумма пополнения должна быть
положительной.")
    self.balance += amount
    print(f"Внесено {amount:.2f} руб. Новый баланс: {self.balance:.2f}
руб.")
```

```

def accrue_interest(self):
    interest = self.balance * (self.interest_rate / 100)
    self.balance += interest
    print(f"Начислено процентов: {interest:.2f} руб. Новый баланс:
{self.balance:.2f} руб.")

def to_dollars(self, rate=90.0):
    return self.balance / rate

def to_euros(self, rate=100.0):
    return self.balance / rate

def amount_in_words(self):
    units = ["", "один", "два", "три", "четыре", "пять", "шесть", "семь",
"восемь", "девять"]
    teens = ["десять", "одиннадцать", "двенадцать", "тринадцать",
"четырнадцать",
        "пятнадцать", "шестнадцать", "семнадцать", "восемнадцать",
"девятнадцать"]
    tens = ["", "", "двадцать", "тридцать", "сорок", "пятьдесят",
        "шестьдесят", "семьдесят", "восемьдесят", "девяносто"]
    hundreds = ["", "сто", "двести", "триста", "четыреста",
        "пятьсот", "шестьсот", "семьсот", "восемьсот", "девятьсот"]

def three_digits_to_words(num):
    if num == 0:
        return ""
    result = []
    if num >= 100:

```

```
    result.append(hundreds[num // 100])
    num %= 100
if 10 <= num < 20:
    result.append(teens[num - 10])
else:
    if num >= 20:
        result.append(tens[num // 10])
        num %= 10
    if num > 0:
        result.append(units[num])
return " ".join(result)
```

```
def num_to_words(n):
```

```
    if n == 0:
        return "ноль"
```

```
    result = []
```

```
    if n >= 1000000:
```

```
        millions = n // 1000000
```

```
        n %= 1000000
```

```
        m_text = three_digits_to_words(millions)
```

```
        if 11 <= millions % 100 <= 19:
```

```
            m_end = "миллионов"
```

```
        elif millions % 10 == 1:
```

```
            m_end = "миллион"
```

```
        elif 2 <= millions % 10 <= 4:
```

```
            m_end = "миллиона"
```

```
        else:
```

```
            m_end = "миллионов"
```

```

result.append(f"{m_text} {m_end}")

if n >= 1000:
    thousands = n // 1000
    n %= 1000
    th_text = three_digits_to_words(thousands)
    th_text = th_text.replace("один", "одна").replace("два", "две")
    if 11 <= thousands % 100 <= 19:
        th_end = "тысяч"
    elif thousands % 10 == 1:
        th_end = "тысяча"
    elif 2 <= thousands % 10 <= 4:
        th_end = "тысячи"
    else:
        th_end = "тысяч"
    result.append(f"{th_text} {th_end}")

if n > 0:
    result.append(three_digits_to_words(n))

return " ".join(result)

```

```

rubles = int(self.balance)
kopecks = int(round((self.balance - rubles) * 100))

```

```

rub_text = num_to_words(rubles)
kop_text = num_to_words(kopecks)

```

```

if 11 <= rubles % 100 <= 19:

```

```
        rub_end = "рублей"
    elif rubles % 10 == 1:
        rub_end = "рубль"
    elif 2 <= rubles % 10 <= 4:
        rub_end = "рубля"
    else:
        rub_end = "рублей"

    if 11 <= kopecks % 100 <= 19:
        kop_end = "копеек"
    elif kopecks % 10 == 1:
        kop_end = "копейка"
    elif 2 <= kopecks % 10 <= 4:
        kop_end = "копейки"
    else:
        kop_end = "копеек"

    return f"{rub_text} {rub_end} {kop_text} {kop_end}"
```

```
if __name__ == '__main__':
    print("--- Демонстрация работы класса Account ---")

    print("\n1. Создание счета:")
    acc = Account("Иванов И.И.", "40817810099910004312", 5.0, 15000.50)
    acc.display()

    print("\n2. Смена владельца:")
    acc.change_owner("Петров П.П.")
```

```
acc.display()
```

```
print("\n3. Пополнение счета:")
```

```
acc.deposit(5000)
```

```
acc.display()
```

```
print("\n4. Снятие средств:")
```

```
acc.withdraw(2000)
```

```
acc.display()
```

```
print("\n5. Начисление процентов:")
```

```
acc accrue_interest()
```

```
acc.display()
```

```
print("\n6. Перевод в валюту:")
```

```
print(f"Сумма в долларах: {acc.to_dollars():.2f} USD")
```

```
print(f"Сумма в евро: {acc.to_euros():.2f} EUR")
```

```
print("\n7. Сумма прописью:")
```

```
print(acc.amount_in_words())
```

```
print("\n8. Ввод нового счета с клавиатуры:")
```

```
new_acc = Account()
```

```
new_acc.read()
```

```
new_acc.display()
```

```
print(f"Сумма прописью: {new_acc.amount_in_words()}")
```

```
1. Создание счета:
Владелец: Иванов И.И.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 15000.50 руб.

2. Смена владельца:
Владелец счета изменен на: Петров П.П.
Владелец: Петров П.П.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 15000.50 руб.

3. Пополнение счета:
Внесено 5000.00 руб. Новый баланс: 20000.50 руб.
Владелец: Петров П.П.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 20000.50 руб.

4. Снятие средств:
Снято 2000.00 руб. Остаток: 18000.50 руб.
Владелец: Петров П.П.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 18000.50 руб.

5. Начисление процентов:
Начислено процентов: 900.03 руб. Новый баланс: 18900.53 руб.
Владелец: Петров П.П.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 18900.53 руб.
```

Рис. 2 — Выполнение программы

```
5. Начисление процентов:
Начислено процентов: 900.03 руб. Новый баланс: 18900.53 руб.
Владелец: Петров П.П.
Номер счета: 40817810099910004312
Процент начисления: 5.0%
Баланс: 18900.53 руб.

6. Перевод в валюту:
Сумма в долларах: 210.01 USD
Сумма в евро: 189.01 EUR

7. Сумма прописью:
восемнадцать тысяч девятьсот рублей пятьдесят три копейки

8. Ввод нового счета с клавиатуры:
Введите данные счета:
Фамилия владельца: Иванов
Номер счета: 796857465
Процент начисления: 2
Сумма в рублях: 123456
Владелец: Иванов
Номер счета: 796857465
Процент начисления: 2.0%
Баланс: 123456.00 руб.
Сумма прописью: сто двадцать три тысячи четыреста пятьдесят шесть рублей ноль копеек
```

Рис. 3 — Выполнение программы

3. Отправили все изменения в удалённый репозиторий на GitHub и удостоверились в отображении всех изменений.

Контрольные вопросы:

1. Как осуществляется объявление класса в языке Python?

Класс объявляется с помощью ключевого слова `class`, за которым следует имя класса и двоеточие. Тело класса записывается с отступом. Имя класса принято писать с заглавной буквы, используя стиль `CamelCase` (например, `MyClass`). Пример:

Листинг кода:

```
class MyClass:
```

```
    var = 10
```

```
def do_smt(self):  
    print("Что-то делаю")
```

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса определяются внутри класса, но вне методов. Они общие для всех экземпляров и доступны как через имя класса, так и через любой экземпляр. Атрибуты экземпляра определяются внутри методов (обычно в `__init__`) через `self.имя = значение`. Они уникальны для каждого объекта. Если атрибут класса неизменяемого типа (число, строка) изменить через экземпляр, создается новый атрибут экземпляра, а атрибут класса останется прежним. Если атрибут класса изменяемого типа (список, словарь) изменить через экземпляр, изменения затронут все экземпляры. Пример:

Листинг кода:

```
class Pet:  
    kind = "mammal"    # атрибут класса  
    pet_names = []     # атрибут класса (изменяемый)  
  
    def __init__(self, name):  
        self.name = name # атрибут экземпляра  
        self.legs = 4    # атрибут экземпляра
```

3. Каково назначение методов класса?

Методы класса определяют поведение объектов. Они похожи на функции, но привязаны к классу и получают первым аргументом `self` — конкретный экземпляр. Методы могут читать и изменять атрибуты экземпляра, вызывать другие методы, возвращать значения. Пример:

Листинг кода:

```
class Ship:  
    def __init__(self, name, capacity):
```

```
self.name = name
self.capacity = capacity
self.cargo = 0
```

```
def load_cargo(self, weight):
    if self.cargo + weight <= self.capacity:
        self.cargo += weight
        print(f"Загружено {weight} тонн")
    else:
        print("Нельзя загрузить столько")
```

```
def free_space(self):
    return self.capacity - self.cargo
```

Также существуют специальные методы с двумя подчёркиваниями по краям (например, `__init__`, `__str__`, `__add__`), которые вызываются автоматически в определённых ситуациях.

4. Для чего предназначен метод `__init__()` класса?

`__init__` — это конструктор. Он вызывается автоматически при создании нового экземпляра класса и служит для инициализации его атрибутов начальными значениями. Если `__init__` не определён, используется конструктор по умолчанию, который ничего не делает. Класс может иметь только один метод `__init__`. Пример:

```
python
class Rational:
    def __init__(self, a=0, b=1):
        if b == 0:
            raise ValueError("Знаменатель не может быть нулём")
        self.numerator = a
        self.denominator = b
```

При создании `Rational(3, 4)` автоматически вызовется `__init__` с аргументами 3 и 4, и будут созданы атрибуты `numerator` и `denominator`.

5. Каково назначение `self`?

`self` — это ссылка на текущий экземпляр класса, через которую метод обращается к атрибутам и другим методам этого экземпляра. В Python `self` указывается явно первым параметром в каждом методе экземпляра. При вызове метода через объект Python автоматически подставляет этот объект в качестве первого аргумента. То есть вызов `volga.get_info()` превращается в `River.get_info(volga)`. Поэтому в определении метода пишут `def get_info(self):`, а при вызове — `volga.get_info()`. Пример:

Листинг кода:

```
class River:
```

```
    def __init__(self, name, length):
```

```
        self.name = name
```

```
        self.length = length
```

```
    def get_info(self):
```

```
        print(f"Длина {self.name} равна {self.length} км")
```

Здесь `self.name` и `self.length` — это атрибуты конкретного экземпляра.

6. Как добавить атрибуты в класс?

В Python атрибуты не объявляются заранее — они создаются в момент присваивания. Есть три способа.

Первый — в конструкторе `__init__` через `self.имя = значение`. Это самый распространённый способ, так атрибут появляется у каждого экземпляра сразу при создании:

Листинг кода:

```
class Ship:
```

```
    def __init__(self, name):
```

```
self.name = name
```

```
self.cargo = 0
```

Второй — в любом другом методе через `self.имя = значение`. Тогда атрибут появится только у тех экземпляров, которые вызвали этот метод:

```
python
```

```
class Ship:
```

```
    def name_captain(self, cap):
```

```
        self.captain = cap
```

Если метод не вызван, при попытке прочесть `self.captain` возникнет ошибка `AttributeError`.

Третий — прямо через экземпляр или класс снаружи: `obj.новый_атрибут = значение` или `ClassName.новый_атрибут = значение`. Это работает, но считается плохим стилем, так как нарушает инкапсуляцию.

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

В Python нет модификаторов доступа вроде `private`, `protected`, `public`. Вместо этого действуют соглашения. Если имя начинается с одного подчёркивания (например, `_width`), это означает «внутренний, не трогай снаружи», но технически доступ к нему есть. Если имя начинается с двух подчёркиваний (например, `__width`), Python применяет искажение имён (`name mangling`): атрибут переименовывается в `_ClassName__width`. Обратиться напрямую как `obj.__width` не получится — будет ошибка, но `obj._ClassName__width` сработает. То есть это не настоящая защита, а соглашение и лёгкая защита от случайного перекрытия имён в наследниках. Пример:

Листинг кода:

```
class Rectangle:
```

```
    def __init__(self, width, height):
```

```
self._width = width    # соглашение: внутренний
self.__height = height # искажение имён
```

```
def get_height(self):
    return self.__height
```

Попытка обратиться к `rect.__height` вызовет ошибку, но `rect._Rectangle__height` сработает.

8. Каково назначение функции `isinstance`?

`isinstance(obj, cls)` проверяет, является ли объект `obj` экземпляром класса `cls` или его наследником. Возвращает `True` или `False`. Функция часто используется для проверки типов аргументов в методах. Пример:

Листинг кода:

```
def add(self, rhs):
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator
        return Rational(a, b)
    else:
        raise ValueError("Операнд должен быть рациональным числом")
```

Здесь проверяется, что `rhs` — объект класса `Rational`. Если да, выполняется сложение. Если нет — выбрасывается исключение. Функция также учитывает наследование: если `rhs` — экземпляр подкласса `Rational`, проверка вернёт `True`.

Вывод: приобрели навыки по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x, узнали о способах создания классов, их реализации, инициализации, методах классов и объектах классов.

